



M.Sc. Business Intelligence & Data Science
Linear Algebra & Numerical Mathematics
Script
Winter Semester 2024

It is prohibited to use the script, even parts of it, without a prior approval by the university outside the ISM.

Responsible for the content of this script is the author or are the authors.

Scripts are not quotable in scientific work.

ISM International School of Management GmbH
Otto-Hahn-Str. 19
44227 Dortmund
www.ism.de

The module “Quantitative Methods” shall provide a basic understanding for all quantitative approaches and applications that are related to data analysis. So this is a direct preparation for the following module “Business Analytics”.

Firstly, this concerns Linear Algebra & Numerical Mathematics as these subjects are essential for all forms of mathematical modeling and related computations.

The course places emphasis on:

- Modelling of commercial problems via systems of linear equations, in particular usage of matrices
- Matrix calculus
- Algorithms for solving systems of linear equations
 - Direct procedures (e.g. Gauß algorithm)
 - Iterative procedures
- Efficiency and performance issues of algorithms
- Computation and approximation topics

Type and duration of exam:

- Written exam (100 Points / 120 minutes)

The course accounts for one third of the exam/module grade.

Lectures of the module	Hours of presence	Hours of self-study	Total workload (h)	SWS
Linear Algebra & Numerical Mathematics (BIDS_03_01)	18	42	60	2
Structure-detecting methods (BIDS_03_02)	18	42	60	2
Structure-validating procedures (BIDS_03_03)	18	42	60	2

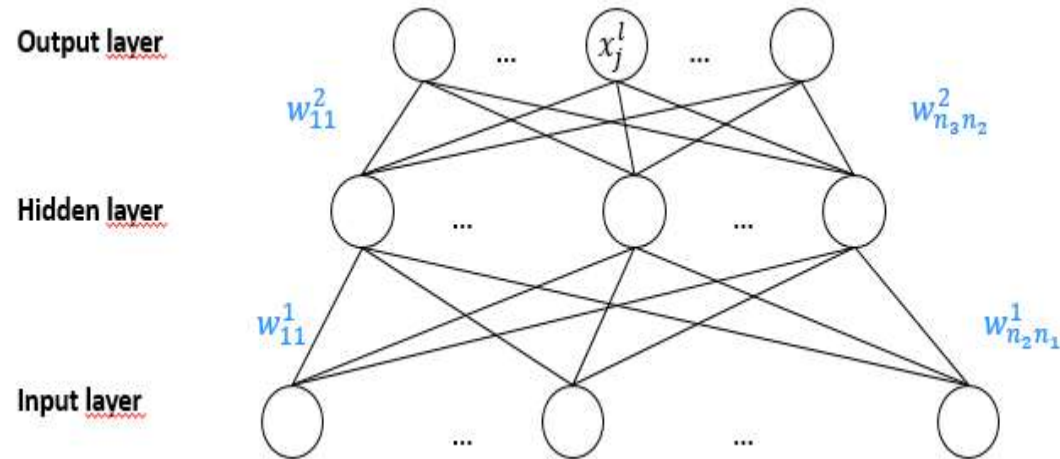
Core text:

- Wendland H. (2017): Numerical linear algebra. Cambridge University Press.
- Hämmerlin G. und Hoffmann K.H. (1991), Numerische Mathematik, Springer
- Barto A. G and Sutton R. S. (2018), „Reinforcement Learning“, MIT Press

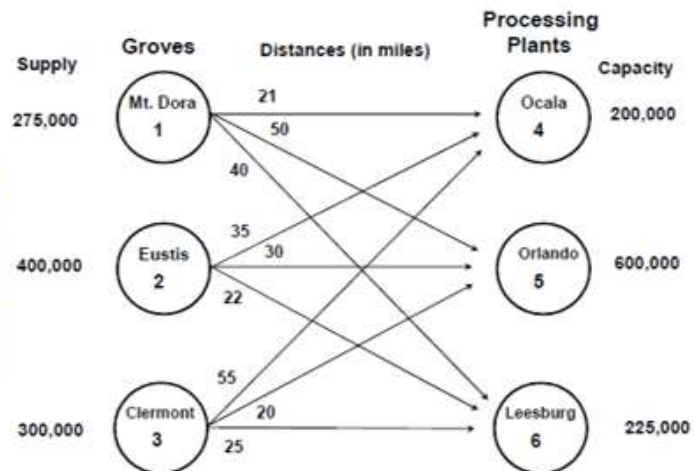
Supplementary literature:

- Strang G. (2016): Introduction to Linear Algebra. Wellesley-Cambridge Press.
- Lay D. C., Lay S. R. and McDonald J. J. (2015): Linear Algebra and Its Applications. 5th Edition. Pearson.
- Jaques I. (2015): Mathematics for Economics and Business. 8th Edition. Pearson.

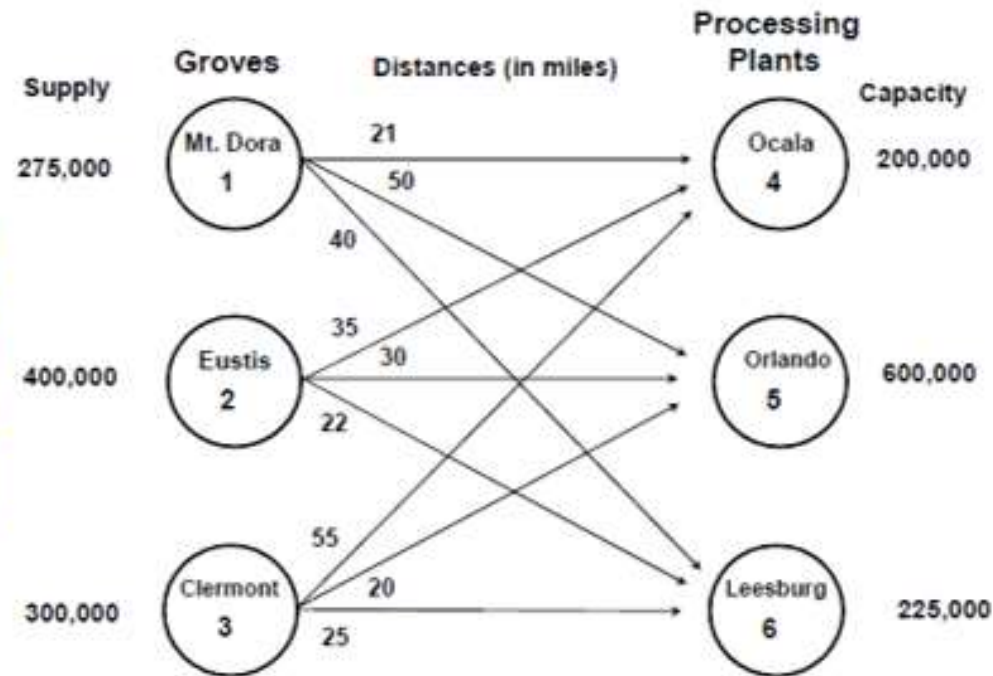
ANNs



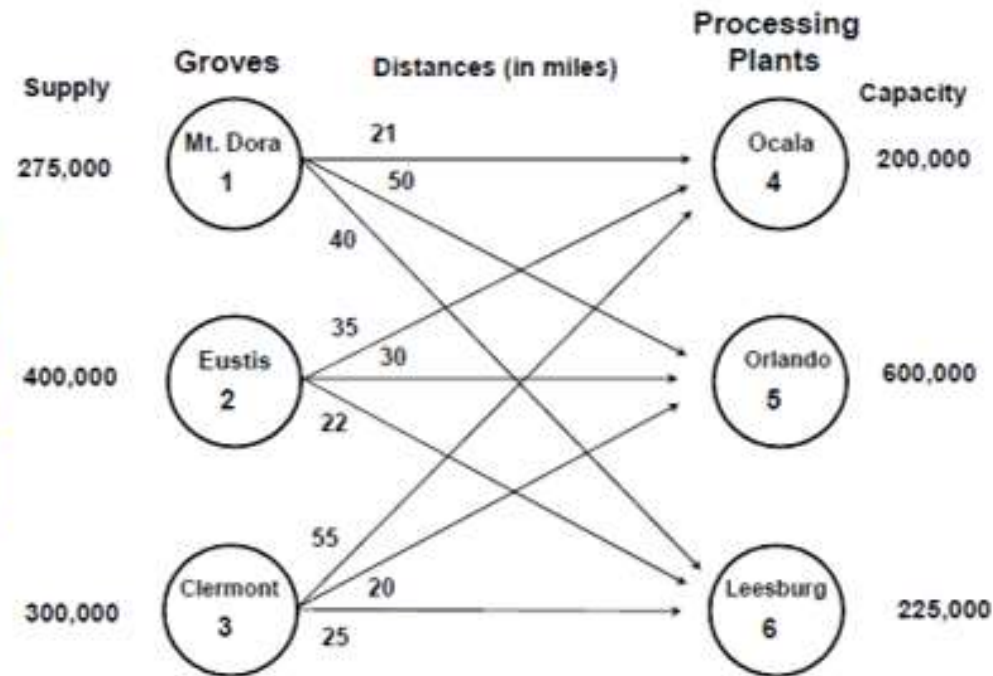
Logistics



Logistics Example



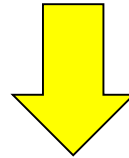
Logistics Example



from / to	Ocala	Orlando	Leesburg
Mt. Dora	21	50	40
Eustis	35	30	22
Clermont	55	20	25

Logistics Example

from / to	Ocala	Orlando	Leesburg
Mt. Dora	21	50	40
Eustis	35	30	22
Clermont	55	20	25

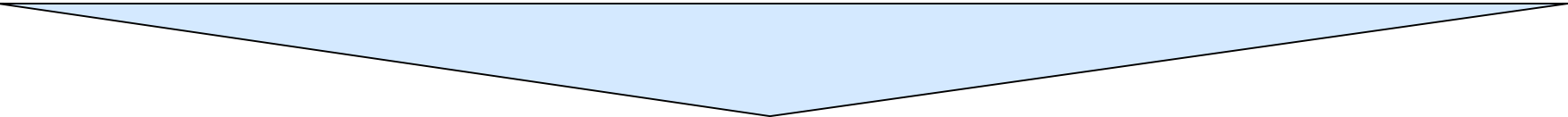


$$\begin{pmatrix} 21 & 50 & 40 \\ 35 & 30 & 22 \\ 55 & 20 & 25 \end{pmatrix}$$

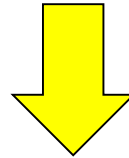
Topic of this course:
how to handle it.

ANNs

Logistics



A single number does not suffice to model the problem!



We need a full array of numbers, a matrix

This leads to Linear Algebra and Numerical Mathematics

1 Linear Algebra

- 1.1 Systems of Linear Equations
- 1.2 Vectors and Matrices
- 1.3 Inverse Matrices
- 1.4 Eigenvalues and Eigenvectors
- 1.5 Gauss Algorithm

2 Numerical Mathematics

- 2.1 Computing
- 2.2 Eigenvalues and Eigenvectors II
- 2.3 Systems of Linear Equations II
- 2.4 Approximation
- 2.5 Iterative Approaches

01

Linear Algebra

- 1.1 Systems of Linear Equations**
- 1.2 Vectors and Matrices
- 1.3 Inverse Matrices
- 1.4 Eigenvalues and Eigenvectors
- 1.5 Gauss Algorithm

Introductory Example:

Blue Ridge Hot Tubs manufactures and sells two models of hot tubs: the Aqua Spa and the Hydro Lux. Howie Jones, the owner and manager of the company, needs to decide how many of each type of hot tub to produce during his next production cycle. Howie buys prefabricated fiberglass hot tub shells from a local supplier and adds the pump and tubing to the shells to create his hot tubs. Howie installs the **one pump in the Aqua Spa and two pumps in the Hydro Lux**. In the warehouse he has **8 pumps**.

From a manufacturing standpoint, the main difference between the two models of hot tubs is the amount of tubing. Each Aqua Spa requires **2 feet of tubing**. Each Hydro Lux requires **5 feet of tubing**. Howie has **18 feet of tubing** in the warehouse. The question is how many Aqua Spas and Hydro Luxes should Howie produce if he wants to fully use all available resources from the warehouse.

Introductory Example - Mathematical Formulation:

Name the (yet unknown) quantities to produce as

x_1 Aqua Spa

x_2 Hydro Lux

There are two types of constraints that restrict the amount of spas that can be build.

$$1x_1 + 2x_2 = 8 \quad \text{Pumps}$$

$$2x_1 + 5x_2 = 18 \quad \text{Tubing}$$

Now how can we easily determine the quantities?

(Remark: This might be an easy problem, however the approach that we use can also be applied to much more difficult problems.)

Introductory Example - Mathematical Formulation:

Now we use matrix-vector notation

$$\begin{pmatrix} 1 & 2 \\ 2 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 8 \\ 18 \end{pmatrix}$$

or even shorter

$$Ax=b.$$



As it will be displayed in the following, the approach of using matrix calculus will lead to a direct solution via the inverse matrix.

Introductory Example - Mathematical Formulation:

Use the inverse matrix A^{-1} on both sides which then yields

$$A^{-1}Ax = A^{-1}b$$

or equivalently

$$x = A^{-1}b.$$

This **directly yields the solution.**

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 5 & -2 \\ -2 & 1 \end{pmatrix} \begin{pmatrix} 8 \\ 18 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$$

Details will be explained later.

So we got the solution in one step. Now, let us consider linear equations and their expression in matrix notation more generally.

1.1 Systems of Linear Equations

Generally the problem could involve much more details (in abstract notation):

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \dots + a_{2n}x_n = b_2$$

...

$$a_{m1}x_1 + a_{m2}x_2 + a_{m3}x_3 + \dots + a_{mn}x_n = b_m$$

And again in notation with matrices:

$$\begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}$$

$$Ax = b$$

This is the basic approach, we still have to discuss the corresponding details.

Exercise:

A firm produces in a two stage production process the two products E1 and E2.

In the first stage the two materials R1 and R2 are processed into intermediate products Z1, Z2 and Z3. For each unit of Z1 five units of R1 and ten units of R2 are needed. For each unit of Z2 two units of R1 and four units of R2 are needed and for each unit of Z3 one unit of both R1 and R2 are needed.

In the second stage the intermediate products are processed into the final products. For the production of one unit of the final product E1 three units of Z2 are needed as well as six units of Z3. For the production of one unit of E2 two units of Z1, two units of Z2 and one unit of Z3 are needed.

Calculate the requirements for the production of both final products.



01

Linear Algebra

- 1.1 Systems of Linear Equations
- 1.2 Vectors and Matrices**
- 1.3 Inverse Matrices
- 1.4 Eigenvalues and Eigenvectors
- 1.5 Gauss Algorithm

Definition

A matrix is a rectangular arrangement of numbers.

Examples for matrices

Vectors (Only one column or row)

Row-vectors:

$$A = (a_1 \quad a_2 \quad a_3) = (5 \quad -3 \quad 7)$$

Column-vector:

$$A = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} = \begin{pmatrix} 7 \\ -2 \\ 9 \end{pmatrix}$$

m x n Matrix (m=rows, n=columns / a_{mn} is the **element** of the matrix in row m and column n)

$$A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix}$$

Quadratic matrix (same number of column and rows)

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = \begin{pmatrix} 5 & -8 \\ 11 & 7 \end{pmatrix} \quad B = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} 6 & 21 & 9 \\ 9 & -1 & 0 \\ -5 & 4 & 13 \end{pmatrix}$$

Diagonal matrix (quadratic, all elements except on the diagonal are zero)

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = \begin{pmatrix} 7 & 0 \\ 0 & -1 \end{pmatrix} \quad B = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} 3 & 0 & 0 \\ 0 & -8 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

(Upper) Triangular matrix (quadratic, all elements below the diagonal are zero)

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = \begin{pmatrix} 7 & 5 \\ 0 & -3 \end{pmatrix} \quad B = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} 6 & 6 & 4 \\ 0 & -1 & 8 \\ 0 & 0 & 13 \end{pmatrix}$$

Identity matrix (quadratic, the diagonal contains only 1, all other elements are zero)

$$E = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$E = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Zero matrix (all elements are zero)

$$\Theta = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

$$\Theta = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Definition: (Transposed matrix)

Transposing a matrix means that each row of the matrix becomes a column of the new (transposed) matrix. Matrix A becomes Matrix A^T .

The first column of a matrix becomes the first row of the new matrix etc. One can remember this that all elements are mirrored on the diagonal.

In particular this means that the row and column number are interchanged.



Example:

$$A = \begin{pmatrix} 7 & 11 & -5 \\ 6 & 8 & 0 \\ 9 & -6 & 12 \end{pmatrix}$$

$$A^T = \begin{pmatrix} 7 & 6 & 9 \\ 11 & 8 & 0 \\ -5 & 0 & 12 \end{pmatrix}$$

$$B = \begin{pmatrix} 7 \\ -2 \\ 9 \end{pmatrix}$$

$$B^T = (7 \quad -2 \quad 9)$$

Exercise:

Transpose the following matrices:

a) $A = \begin{pmatrix} 8 & -3 & 11 \\ -9 & 0 & 1 \\ 8 & 4 & 1 \end{pmatrix}$

b) $B = \begin{pmatrix} -4 & 0 \\ 11 & 5 \end{pmatrix}$

c) $C = (7 \quad -2 \quad 8)$

d) $D = \begin{pmatrix} 3 & 8 \\ 0 & -5 \\ 9 & -3 \end{pmatrix}$

Matrix addition and subtraction:

It is possible to add or subtract matrices of the same form ($m \times n$). In these cases the respective numbers are added or subtracted. Both operations can only work if both matrices have the same number of rows and the same number of columns.

$$\begin{pmatrix} 5 & 1 \\ 7 & 11 \end{pmatrix} + \begin{pmatrix} 6 & 5 \\ 3 & -6 \end{pmatrix} = \begin{pmatrix} 5+6 & 1+5 \\ 7+3 & 11+(-6) \end{pmatrix} = \begin{pmatrix} 11 & 6 \\ 10 & 5 \end{pmatrix}$$

$$\begin{pmatrix} 7 & 21 & 9 \\ 4 & -7 & 4 \\ 22 & 8 & -13 \end{pmatrix} - \begin{pmatrix} -5 & 3 & 5 \\ 8 & 23 & 8 \\ 12 & 7 & 0 \end{pmatrix} = \begin{pmatrix} 7-(-5) & 21-3 & 9-5 \\ 4-8 & -7-23 & 4-8 \\ 22-12 & 8-7 & -13-0 \end{pmatrix} = \begin{pmatrix} 12 & 18 & 4 \\ -4 & -30 & -4 \\ 10 & 1 & -13 \end{pmatrix}$$

Exercise:

Calculate the following expressions by the using the given matrices:

$$A = \begin{pmatrix} 3 & -6 \\ -9 & 12 \\ 5 & 0 \end{pmatrix}$$

$$B = (6 \quad -3 \quad 12)$$

$$C = \begin{pmatrix} 6 & -5 \\ 3 & -9 \\ -2 & 11 \end{pmatrix}$$

$$D = \begin{pmatrix} 7 & -3 & 4 \\ 9 & 21 & -5 \end{pmatrix}$$

a) $A + C$

b) $C + D^T$

c) $D^T - (A + C)$

d) $C - B$

Multiplication of a matrix with a number:

$$r \cdot \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = \begin{pmatrix} r \cdot a_{11} & r \cdot a_{12} \\ r \cdot a_{21} & r \cdot a_{22} \end{pmatrix}$$

Example:

$$3 \cdot \begin{pmatrix} 3 & -7 \\ 5 & 2 \end{pmatrix} = \begin{pmatrix} 9 & -21 \\ 15 & 6 \end{pmatrix}$$

Multiplication of two matrices

Requirement: The product $A \cdot B$ can only be calculated if:

Number of columns in A = Number of rows in B

Dimension of the product = Number of rows in A x Number of columns in B

$$A \cdot B = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{pmatrix}$$
$$= \begin{pmatrix} a_{11} \cdot b_{11} + a_{12} \cdot b_{21} & a_{11} \cdot b_{12} + a_{12} \cdot b_{22} & a_{11} \cdot b_{13} + a_{12} \cdot b_{23} \\ a_{21} \cdot b_{11} + a_{22} \cdot b_{21} & a_{21} \cdot b_{12} + a_{22} \cdot b_{22} & a_{21} \cdot b_{13} + a_{22} \cdot b_{23} \\ a_{31} \cdot b_{11} + a_{32} \cdot b_{21} & a_{31} \cdot b_{12} + a_{32} \cdot b_{22} & a_{31} \cdot b_{13} + a_{32} \cdot b_{23} \end{pmatrix}$$



Example:

$$A \cdot B = \begin{pmatrix} 6 & 4 \\ 1 & -7 \\ 3 & 9 \end{pmatrix} \cdot \begin{pmatrix} 2 & 7 & 3 \\ 1 & 5 & 8 \end{pmatrix} = \begin{pmatrix} 6 \cdot 2 + 4 \cdot 1 & 6 \cdot 7 + 4 \cdot 5 & 6 \cdot 3 + 4 \cdot 8 \\ 1 \cdot 2 + (-7) \cdot 1 & 1 \cdot 7 + (-7) \cdot 5 & 1 \cdot 3 + (-7) \cdot 8 \\ 3 \cdot 2 + 9 \cdot 1 & 3 \cdot 7 + 9 \cdot 5 & 3 \cdot 3 + 9 \cdot 8 \end{pmatrix} =$$
$$\begin{pmatrix} 16 & 62 & 50 \\ -5 & -28 & -53 \\ 15 & 66 & 81 \end{pmatrix}$$

Falk's Schematic:

2	7	3
1	5	8

6	4
---	---

1	-7
---	----

3	9
---	---

Exercise:

Use the given matrices to calculate the expressions given below:

$$A = \begin{pmatrix} 3 & -9 \\ 2 & 5 \end{pmatrix} \quad B = \begin{pmatrix} 8 & 6 \\ 0 & 1 \end{pmatrix} \quad C = \begin{pmatrix} 0 & -5 & 8 \\ 7 & 3 & 0 \\ -3 & 7 & 1 \end{pmatrix} \quad D = \begin{pmatrix} 5 \\ -7 \\ 3t \end{pmatrix} \quad F = \begin{pmatrix} 4 & -8 \\ 2 & 7 \\ -5 & 1 \end{pmatrix}$$

- a) $A \cdot B$
- b) $5 \cdot B - A$
- c) $B \cdot A$
- d) $8 \cdot (A - B)^2$
- e) $A \cdot F$
- f) $C \cdot D$
- g) $B \cdot F^T$
- h) $(A - B) \cdot F^T$

Norms for vectors and matrices

As matrices (vectors being special cases of them) are high dimensional objects, it is sometimes **useful** to have a **description by just a single number**. The **norm** („*magnitude*“) that we introduce now, will be helpful later for assessing computational errors.

Definition of a norm on a vector space

Let X be a vector space over some field F (for example the real numbers). Then a **norm** on this **vector** space is a **mapping**

$$\|\cdot\|: X \rightarrow F, x \rightarrow \|x\|$$

which fulfills for all x and y of X and all α of F :

$$(i) \quad \|x\| = 0 \Leftrightarrow x = 0$$

$$(ii) \quad \|\alpha x\| = |\alpha| \|x\|$$

$$(iii) \quad \|x + y\| \leq \|x\| + \|y\|$$

Norms for vectors and matrices

The most easy example for a vector norm is the **Euclidean Norm**:

$$\|\cdot\|: R^n \rightarrow R, x \rightarrow \sqrt{x_1^2 + \dots + x_n^2}$$

It fulfills all three characteristics:

$$(i) \|x\| = 0 = \sqrt{0^2 + \dots + 0^2} \Leftrightarrow x = 0$$

$$(ii) \|\alpha x\| = \sqrt{(\alpha x_1)^2 + \dots + (\alpha x_n)^2} = \sqrt{\alpha^2(x_1^2 + \dots + x_n^2)} = |\alpha| \sqrt{x_1^2 + \dots + x_n^2} = |\alpha| \|x\|$$

$$(iii) \|x + y\| = \sqrt{(x_1 + y_1)^2 + \dots + (x_n + y_n)^2} \leq \|x\| + \|y\| = \sqrt{x_1^2 + \dots + x_n^2} + \sqrt{y_1^2 + \dots + y_n^2}$$

(some computations)

Exercise:

Calculate the Euclidean norm of:

1) $\begin{pmatrix} 7 \\ -3 \\ 8 \end{pmatrix}$

2) $\begin{pmatrix} 7 \\ 3 \\ 8 \end{pmatrix}$

3) $\begin{pmatrix} 2 \\ 5 \\ -7 \end{pmatrix}$

Norms for vectors and matrices

To use this concept also for matrices one has to simply observe that matrices are „larger“ vectors. As a matrix multiplication is just a sum of products there exists always a constant C such that

$$\|Ax\| \leq C\|x\| \text{ for all } x$$

Hence we will define the **norm** (the biggest such constant) **of a matrix** as follows:

$$\|A\| := \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}$$

So now we can always make the estimate

$$\|Ax\| \leq \|A\| * \|x\|$$

to control the result of a matrix vector multiplication.

As we have derived a matrix norm from a given vector norm, the matrix norm is called an **induced norm**. This also means that by using a different vector norm as a starting point, we will get a different induced matrix norm.

01

Linear Algebra

- 1.1 Systems of Linear Equations
- 1.2 Vectors and Matrices
- 1.3 Inverse Matrices**
- 1.4 Eigenvalues and Eigenvectors
- 1.5 Gauss Algorithm

Motivation

We have already noted that an inverse matrix can lead to a direct solution of a system of linear equations. Now we provide more details about them, in particular how we can find them.

Definition of the inverse matrix:

$$A \cdot A^{-1} = A^{-1} \cdot A = E \quad (E = \text{Identity})$$

If matrix A is multiplied with A^{-1} , it results in the identity (matrix). A^{-1} is thus the inverse matrix of A . The inverse matrix is **unique**. A cannot have two or more inverse matrices. Different matrices have different inverse matrices.

Only quadratic matrices can be inverted. Rectangular, non-quadratic matrices are not invertable. But also **not every quadratic** matrix is invertable.

Calculation of the inverse matrix:

$$A = \begin{pmatrix} 2 & 2 \\ 1 & -1 \end{pmatrix}$$

1. Step: The matrix is extended by the identity on the right-hand side:

$$\left(\begin{array}{cc|cc} 2 & 2 & 1 & 0 \\ 1 & -1 & 0 & 1 \end{array} \right)$$

2. Step: Now try to reformulate the matrix so that the identity stands on the left-hand side. The following operations can be used:

- Switch two rows
- Multiplication or Division of a row with a real number ($\neq 0$)
- Addition or Subtraction of a row to /from a second row

3. Step: If the situation occurs that one row only contains zeros the matrix is not invertible.



Example:

$$\left(\begin{array}{cc|cc} 2 & 2 & 1 & 0 \\ 1 & -1 & 0 & 1 \end{array}\right) \quad 0,5 \cdot (I)$$

$$\left(\begin{array}{cc|cc} 1 & 1 & 0,5 & 0 \\ 1 & -1 & 0 & 1 \end{array}\right) \quad (II) - (I)$$

$$\left(\begin{array}{cc|cc} 1 & 1 & 0,5 & 0 \\ 0 & -2 & -0,5 & 1 \end{array}\right) \quad -0,5 \cdot (II)$$

$$\left(\begin{array}{cc|cc} 1 & 1 & 0,5 & 0 \\ 0 & 1 & 0,25 & -0,5 \end{array}\right) \quad (I) - (II)$$

$$\left(\begin{array}{cc|cc} 1 & 0 & 0,25 & 0,5 \\ 0 & 1 & 0,25 & -0,5 \end{array}\right)$$

$$A^{-1} = \begin{pmatrix} 0,25 & 0,5 \\ 0,25 & -0,5 \end{pmatrix}$$

Trying to establish the **identity matrix** on the left side in a **counter-clockwise** manner always finishes in **at most four steps**. Bigger matrices need a slightly different approach.

Applying the inverse matrix to a system of linear equations:

$$2x_1 + 2x_2 = 2$$

$$x_1 - x_2 = 1$$

Rephrasing:

$$\begin{pmatrix} 2 & 2 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Matrix A is inverted:

$$A^{-1} = \begin{pmatrix} 0,25 & 0,5 \\ 0,25 & -0,5 \end{pmatrix}$$

Resulting in:

$$A^{-1}Ax = A^{-1}b \quad \Rightarrow \quad Ex = A^{-1}b \quad \Rightarrow \quad x = A^{-1}b$$

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0,25 & 0,5 \\ 0,25 & -0,5 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ and thus } x_1 = 1 \text{ and } x_2 = 0$$

Exercise:

Calculate, if possible, the inverse of the following matrices:

a) $A = \begin{pmatrix} 2 & 2 \\ 4 & 5 \end{pmatrix}$

b) $B = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}$

c) $C = \begin{pmatrix} 1 & 2 & 0 \\ 1 & 0 & 1 \\ 0 & 3 & -2 \end{pmatrix}$

d) $D = \begin{pmatrix} 3 & 8 & -9 \\ 2 & -5 & 1 \end{pmatrix}$

e) $E = \begin{pmatrix} 0 & 1 & -2 \\ 1 & 1 & 0 \\ 2 & 0 & 2 \end{pmatrix}$

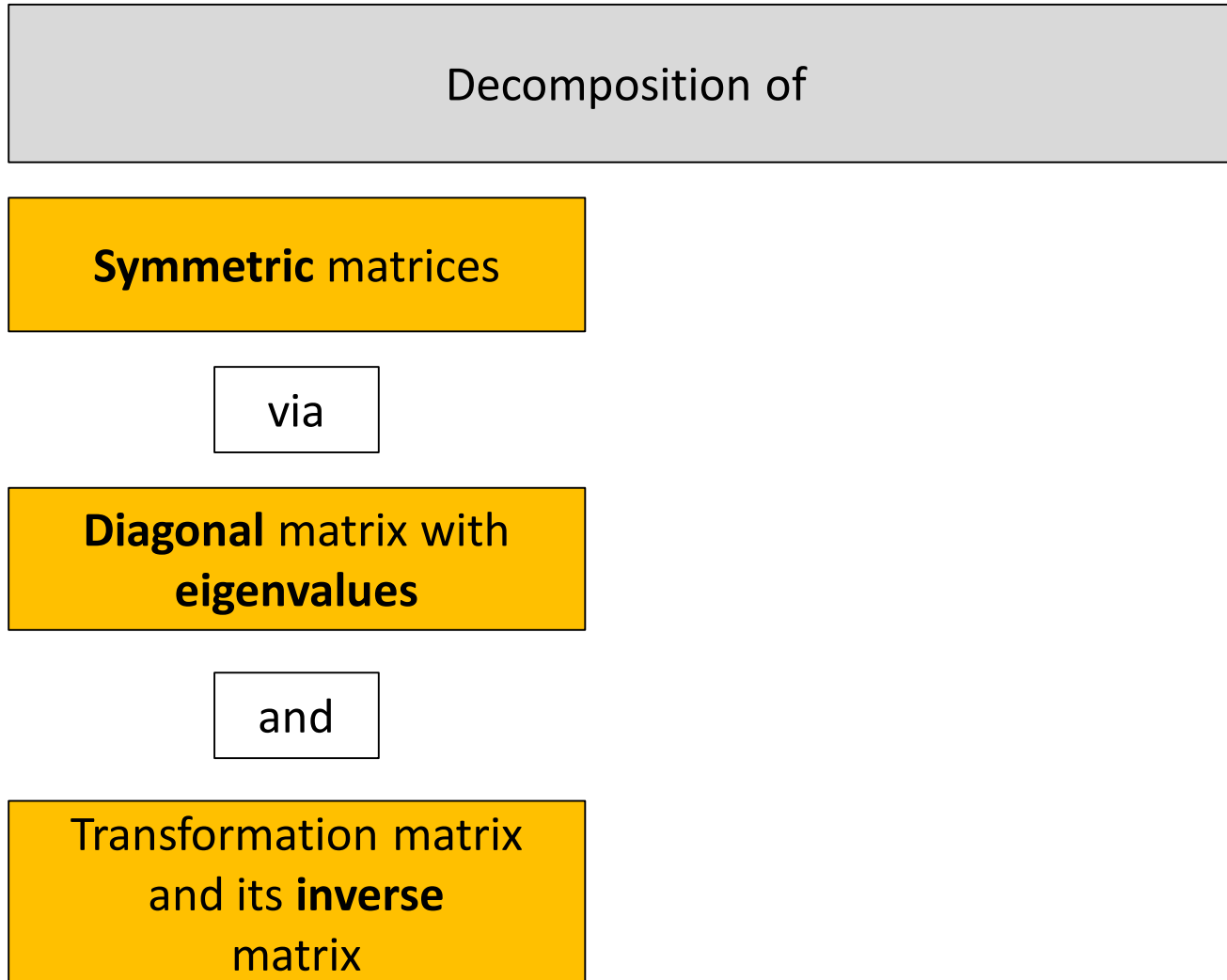
f) $F = \begin{pmatrix} 4 & 4 \\ 2 & 2 \end{pmatrix}$

01

Linear Algebra

- 1.1 Systems of Linear Equations
- 1.2 Vectors and Matrices
- 1.3 Inverse Matrices
- 1.4 Eigenvalues and Eigenvectors**
- 1.5 Gauss Algorithm

Introduction



What is a symmetric matrix?

$$A^T = A$$

This means that it is symmetric with respect to the diagonal, for example $A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$.

The **decomposition** into a product involving a diagonal matrix is **guaranteed** for **symmetric** matrices, but may also be possible for non-symmetric matrices.

Digression: Some useful rules for matrix computation

- Transposing **twice** does **not change** anything.

$$A^{TT} = A$$

- **Transposing** changes the **order of a multiplication** since the dimension have to fit.

$$(ABCD)^T = D^T C^T B^T A^T$$

- When dealing with several multiplications, you can **start anywhere** to get the same result.

$$ABCD = (\mathbf{AB})CD = A(\mathbf{BC})D = AB(\mathbf{CD})$$

1.4 Eigenvalues and Eigenvectors

As already displayed, multiplication of a matrix with a vector yields another vector:

$$Ax = b$$

If we have the special case of a quadratic matrix, there exist very special vectors, the **eigenvectors**. They are essential for certain statistical procedures as well as other applications (computer graphics) and have the property that they are exactly mapped to a **multiple** of themselves:

$$Av = \gamma v$$

Example:

$$A = \begin{pmatrix} 2 & -4 \\ -1 & -1 \end{pmatrix}, v_1 = \begin{pmatrix} -4 \\ 1 \end{pmatrix}, v_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$Av_1 = \begin{pmatrix} -12 \\ 3 \end{pmatrix} = 3v_1$$

$$Av_2 = \begin{pmatrix} -2 \\ -2 \end{pmatrix} = -2v_2$$

1.4 Eigenvalues and Eigenvectors

In the given case (**generally, when the matrix is symmetric**), the eigenvectors can even be further used to give a decomposition of the matrix A in terms of

- an **eigenvector matrix**,
- an **eigenvalue matrix**
- the **inverse** of the **eigenvector matrix**:

$$A = (v_1 \quad v_2) \begin{pmatrix} \gamma_1 & 0 \\ 0 & \gamma_2 \end{pmatrix} (v_1 \quad v_2)^{-1}$$

A typical use case is the factor analysis (→ Structure-Detecting Methods).

Example:

$$\begin{pmatrix} 2 & -4 \\ -1 & -1 \end{pmatrix} = \begin{pmatrix} -4 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & -2 \end{pmatrix} \begin{pmatrix} -0,2 & 0,2 \\ 0,2 & 0,8 \end{pmatrix}$$

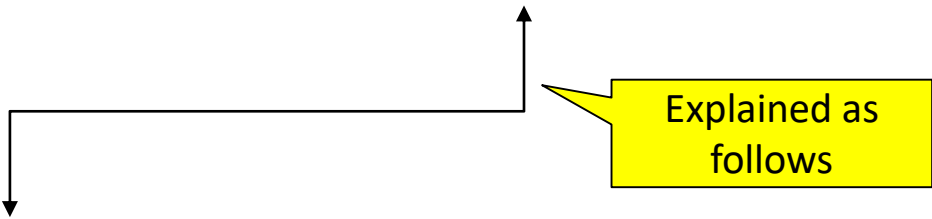
1.4 Eigenvalues and Eigenvectors

But how can the eigenvalues and eigenvectors be found?

For the eigenvalues one has to solve

$$|A - \gamma I| = \begin{vmatrix} a_{11} - \gamma & a_{12} \\ a_{21} & a_{22} - \gamma \end{vmatrix} = 0$$

Example:


$$\begin{vmatrix} 2 - \gamma & -4 \\ -1 & -1 - \gamma \end{vmatrix} = (2 - \gamma)(-1 - \gamma) - (-1)(-4) = \gamma^2 - \gamma - 6 = 0$$

$$(\gamma - 3)(\gamma + 2) = 0$$

$$\gamma_1 = 3, \gamma_2 = -2$$

1.4 Eigenvalues and Eigenvectors

After having found the eigenvalues, the corresponding eigenvectors have to be found as vectors which fulfill

$$(A - \lambda_1 I)v_1 = 0$$

$$(A - \lambda_2 I)v_2 = 0$$

Example:

$$\begin{pmatrix} 2 - 3 & -4 \\ -1 & -1 - 3 \end{pmatrix} v_1 = \begin{pmatrix} -1 & -4 \\ -1 & -4 \end{pmatrix} v_1 = 0$$

$$v_1 = \begin{pmatrix} -4 \\ 1 \end{pmatrix} \text{ (for example)}$$

$$\begin{pmatrix} 2 + 2 & -4 \\ -1 & -1 + 2 \end{pmatrix} v_2 = \begin{pmatrix} 4 & -4 \\ -1 & 1 \end{pmatrix} v_2 = 0$$

$$v_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \text{ (for example).}$$

Taking all together, one arrives at

$$A = (v_1 \quad v_2) \begin{pmatrix} \gamma_1 & 0 \\ 0 & \gamma_2 \end{pmatrix} (v_1 \quad v_2)^{-1}$$

$$\begin{pmatrix} 2 & -4 \\ -1 & -1 \end{pmatrix} = \begin{pmatrix} -4 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & -2 \end{pmatrix} \begin{pmatrix} -0,2 & 0,2 \\ 0,2 & 0,8 \end{pmatrix}$$

Interesting question:

We have observed that we have a certain flexibility for choosing the eigenvectors.

Doesn't the particular **choice** of eigenvectors lead to a **totally different** diagonal matrix **decomposition**?

Answer: **No!**

Why?

Any „**arbitrary**“ **scaling factor** in the eigenvector matrix is „**balanced**“ by the respective inverse matrix in the overall product. Important: The identified diagonal matrix is always the same (hence unique).

If you doubt it, just make a simple computation to convince yourself (one alternative example is even given in the official solutions).

Exercise

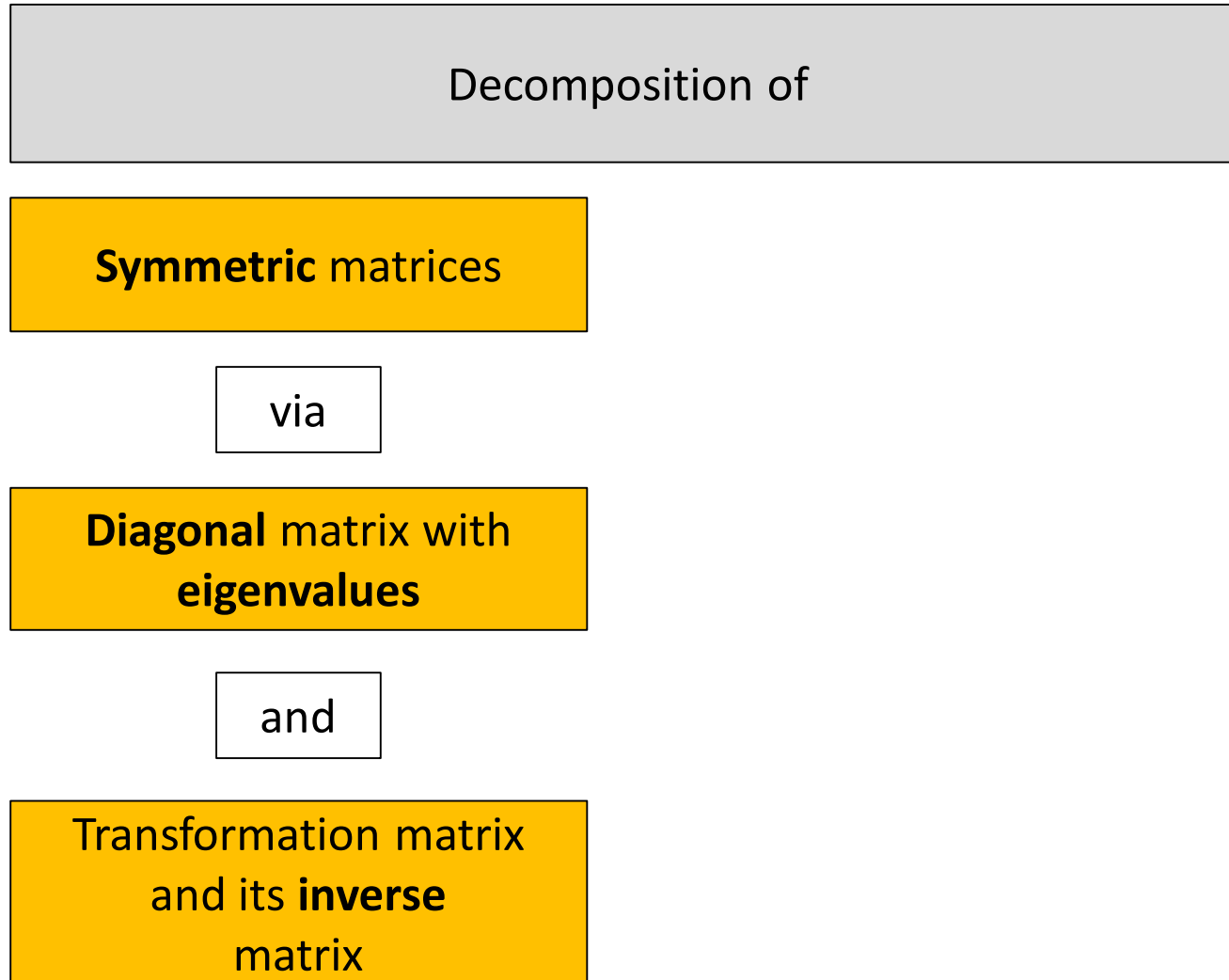
Decompose the following matrices in terms of a diagonal matrix:

1) $\begin{pmatrix} -3 & 4 \\ 4 & 3 \end{pmatrix}$

2) $\begin{pmatrix} 11 & -6 \\ 18 & -10 \end{pmatrix}$

3) $\begin{pmatrix} -8 & -30 \\ 4 & 14 \end{pmatrix}$

Summary



01

Linear Algebra

- 1.1 Systems of Linear Equations
- 1.2 Vectors and Matrices
- 1.3 Inverse Matrices
- 1.4 Eigenvalues and Eigenvectors
- 1.5 Gauss Algorithm**

General idea:

The inverse matrix helps us in finding the solution to a system of linear equations.

However, as the **size** of the **matrix grows** the computations increase a lot hence **slowing down computational efforts considerably** (even for automated procedures).

Therefore one might ask if there exist a **more efficient procedure** under computational considerations.

There exists such a procedure. The idea is, unlike for the case of the inverse matrix, to assess **not the full matrix** resulting from the system of linear equations, but only the **lower triangle** of it.

Gauss-Algorithm:

First write the system of linear equations in matrix notation:

$$\begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}$$

Then rewrite the system of linear equations in short form:

$$\left(\begin{array}{ccc|c} a_{11} & \cdots & a_{1n} & b_1 \\ \vdots & \ddots & \vdots & \vdots \\ a_{m1} & \cdots & a_{mn} & b_m \end{array} \right)$$



Do the aforementioned row manipulations until the **lower triangle** equals the identity matrix:

$$\left(\begin{array}{cccc|c} 1 & ? & \cdots & ? & c_1 \\ 0 & & & \vdots & \vdots \\ \vdots & & & ? & \vdots \\ 0 & \cdots & 0 & 1 & c_m \end{array} \right)$$

The values c_i of the right side deliver the solution to the system of linear equations.

Gauss-Algorithm:

After translating the short-hand notation

$$\left(\begin{array}{cccc|c} 1 & ? & \dots & ? & c_1 \\ 0 & & & \vdots & \vdots \\ \vdots & & & ? & \vdots \\ 0 & \dots & 0 & 1 & c_m \end{array} \right)$$

back into linear equations,

$$\begin{aligned} 1 * x_1 + ? * x_2 + \dots + 1 * x_{n-1} + ? * x_n &= c_1 \\ 0 * x_1 + 1 * x_2 + \dots + ? * x_{n-1} + ? * x_n &= c_2 \\ \dots & \\ 0 * x_1 + 0 * x_2 + \dots + 1 * x_{n-1} + ? * x_n &= c_{m-1} \\ 0 * x_1 + 0 * x_2 + \dots + 0 * x_{n-1} + 1 * x_n &= c_m \end{aligned}$$

one can then solve the equations in each row very easily by starting from the bottom:

- It is immediately clear that $x_n = c_m$
- One can then substitute x_n in the equation one line above which then yields x_{n-1}
- Then one continues in this manner to obtain $x_{n-2}, x_{n-3}, \dots, x_1$

Gauss-Algorithm:

Computational assessment:

From a computational point of view, the Gauss algorithm is very similar to finding the inverse matrix, however here one **only** manipulates the **lower triangular half**.

This might not be decisive when one deals with small matrices, however it is much more efficient if **large matrices** are concerned as one **saves a lot of computations** (that one would otherwise do for finding the inverse matrix).

So for **cases** where one does **not exactly need the inverse matrix** (unlike in Markowitz Portfolio Theory for example), the **Gauss algorithm** constitutes a **more efficient** procedure for finding the solution of the system of linear equations.

Gauss-Algorithm: (Solutions)

There are **three possibilities for the solution set** of a system of linear equations :

Exactly one solution:

The non zero rows yield exactly one solution.

$$\left(\begin{array}{cc|c} 1 & 0 & 5 \\ 0 & 1 & 7 \\ 0 & 0 & 0 \end{array}\right)$$



No solution:

There is at least one row with only zeros on the **left** and on the **right** the number is different from zero.

$$\left(\begin{array}{cc|c} 1 & 0 & 5 \\ 0 & 1 & 7 \\ 0 & 0 & 3 \end{array}\right)$$

Gauss-Algorithm: (Solutions)

Infinitely many solutions:

The equations of the non zero rows are fulfilled by arbitrarily many solutions.

$$\left(\begin{array}{cc|c} 1 & 1 & 5 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{array} \right)$$

The solutions can be written in the following way:

$$x_1 + x_2 = 5 \Rightarrow x_1 = -x_2 + 5$$

and thus:

$$x_1 = -x_2 + 5$$

$$x_2 = x_2$$

or:

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_2 \begin{pmatrix} -1 \\ 1 \end{pmatrix} + \begin{pmatrix} 5 \\ 0 \end{pmatrix} \text{ for any real } x_2$$

Example 16:

Solve the following linear systems using the Gauss-Algorithm:

$$a) \begin{pmatrix} 2 & 2 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

$$b) \begin{pmatrix} 2 & -5 & 3 \\ 4 & -12 & 8 \\ 3 & 1 & -2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \\ 9 \end{pmatrix}$$

$$c) \begin{aligned} x_1 - 2x_3 &= -4 \\ x_1 + x_2 &= 2 \\ 2x_1 + 2x_3 &= 4 \end{aligned}$$

01

Numerical Mathematics

2.1 Computing

2.2 Eigenvalues and Eigenvectors II

2.3 Systems of Linear Equations II

2.4 Approximation

2.5 Iterative Approaches

General Idea

Here we display and discuss several aspects that do not only apply to numerical linear algebra, but to algorithms and computations in general.

We consider:

- Representations of numbers
- Error analyses
- Algorithms (in particular order of complexity)

Representations of numbers

Normally, all numbers are displayed with respect to a base of 10.

Example:

$$\begin{aligned} 514 &= 500 + 10 + 4 = \\ &= 5 * 100 + 1 * 10 + 4 * 1 = 5 * 10^2 + 1 * 10^1 + 4 * 10^0 \end{aligned}$$

However, computer only understand digital numbers, implying a base of 2!

Example (continued):

$$\begin{aligned} 514_{10} &= 500 + 10 + 4 = \\ &= 1 * 512 + 1 * 2 = 1 * 2^9 + 1 * 2^1 = 1000000010_2 \end{aligned}$$

Representations of numbers

From the example one can easily infer that a lot of different representations of numbers are possible with respect to different power bases.

The most common are

- Dual (Digits): 2
- Octal: 8
- Decimal: 10
- Hexadecimal: 16

Representations of numbers - Exercise

Write the following numbers in a digital basis:

1) 324_{10}

2) 137_{10}

Error analyses

When doing a structured analysis of errors, one realizes that there are typical three different types of errors:

- Data errors
- Procedural errors
- Rounding errors

Error analyses

When doing a structured analysis of errors, one realizes that there are typically three different types of errors:

- Data errors:

This refers to **mistakes in the input** that constitutes the start of the computations.

- Procedural errors:

Sometimes certain **mathematical expressions** have to be **replaced by others** to guarantee a more efficient computation. Thereby **imprecisions** are taken into account.

- Rounding errors:

Numbers on computers are only stored in a **constrained number of decimals**. **Changes** that would be **necessary** due to further decimals are just **omitted**.

Error analyses

- Procedural errors:

Sometimes certain **mathematical expressions** have to be **replaced by others** to guarantee a more efficient computation. Thereby **imprecisions** are taken into account.

Example:

Instead of considering the differential quotient, one uses the difference quotient if the differential quotient (= derivative) would be extremely difficult. A simple example to show the issue: For instance, when one wants to evaluate the slope of the function $f(x) = x^2$ at $x=4$:

- Differential quotient: The differential quotient is the derivative. As $2x$ is the derivative of x^2 , the slope is exactly $2 * 4 = 8$
- Difference quotient: One considers $(f(b) - f(4))/(b - 4)$ for b „close“ to 4. Take $b=4,001$ to get $8,001$.

Almost the same solution, but not exactly the same due to a different procedure.

Error analyses

- Rounding errors:

Numbers on computers are only stored in a **constrained number of decimals**. **Changes** that would be **necessary** due to further decimals are just **omitted**.

Example:

We just consider how roundings impact the result of computations:
we want to add the three numbers 1.4 and 4.3 and 3.4 .

- If we round each number before adding them, we get 1, 4 and 3 with a total result of 8.
- However, if the sum them up first (result 9.1) and then round, we get 9 which different.

Therefore, we see that roundings may have a big impact on the final result.

Algorithms

Algorithms are simply **sequences of commands**. They are used nowadays in many places for many purposes, but they are **not really new**. Consider for example **Euclid's algorithm** for finding the biggest common divisor of two integer numbers **from 325 B.C.(!)**.

Input: Two integer numbers m and n (big number and small number)

Output: Biggest common divisor (again an integer number)

Procedure: $m' := m, n' := n$

1) Compute $m' : n'$ (integer division with remainder).

Let the rest of the division be denoted as r (of course $r < n'$).

2) Is $r = 0$?

If YES then the greatest common divisor is n' . DONE

ELSE $m' := n', n' := r$ and continue from the beginning.

Algorithms

Euclid's algorithm: Example (two iterations)

Input: Two integer numbers 35 and 28

Output: Biggest common divisor (again an integer number)

Procedure: $m' := 35, n' := 28$

1) Compute $m' : n' = 35 : 28 = 1$. Let the rest of the division be denoted as $r = 7$ (of course $7 < 28$).

2) Is $r = 0$? **No!**

~~If YES then the greatest common divisor is n^* . DONE~~

~~ELSE $m' := 28, n' := 7$ and Continue from the beginning.~~

1) Compute $m' : n' = 28 : 7 = 4$. Let the rest of the division be denoted as $r = 0$ (of course $0 < 7$).

2) Is $r = 0$? **Yes!**

If YES then the greatest common divisor is 7. **DONE**

~~ELSE $m^* := 28, n^* := 7$ and continue from the beginning.~~

Algorithms

The **order of complexity** of an algorithm is also very important. The order of complexity evaluates **how many computations based on the input** are needed (evaluated in terms of a function).

Simple example:

- 1) **Search for maximum and minimum:** Given a list of n numbers, the algorithm has to check each of those n numbers, hence the complexity is **linear**.
- 2) **Matrix multiplication:** for every entry of a matrix (typically $n \times n$) one has to do a computation involving $2n$ numbers. Hence the order of complexity is **cubic**.

Here linear and cubic are examples of polynomials. Algorithms that can be solved by such a dependency are said to belong to the class that can be solved in **polynomial time**.

Very complex problems from graph/network theory cannot be solved in this manner, indicating that they are hard to solve, posing even a challenge to „supercomputers“ (called „**NP-complete**“).

Algorithms

An algorithm always has the same structure (with different content however):

- Input: The input describes what data is needed **to get the procedure started**. Usually a lot of numbers in a table, but also text, pictures, sound etc.
- Output: The output describes the **goal that is to be achieved** after completion of all computations and other commands. Is usually strongly related to the input (classification of pictures, computations etc.)
- Procedure: This is then the main part which describes how the input is transformed into the output. As input and output can be very different according to individual needs, the main parts of algorithms **can be very diverse** and make use of very distinct methods.

Algorithms

An algorithm can be assessed along the following dimensions:

- Definiteness:
Each **step** from the main procedural part must be clearly defined **without any ambiguity**.
- Finiteness:
The algorithm should be executable in a **specified amount of time**.
- Generality:
The algorithm should be applicable **to all problems within a certain class** of problems.

01

Numerical Mathematics

2.1 Computing

2.2 Eigenvalues and Eigenvectors II

2.3 Systems of Linear Equations II

2.4 Approximation

2.5 Iterative Approaches

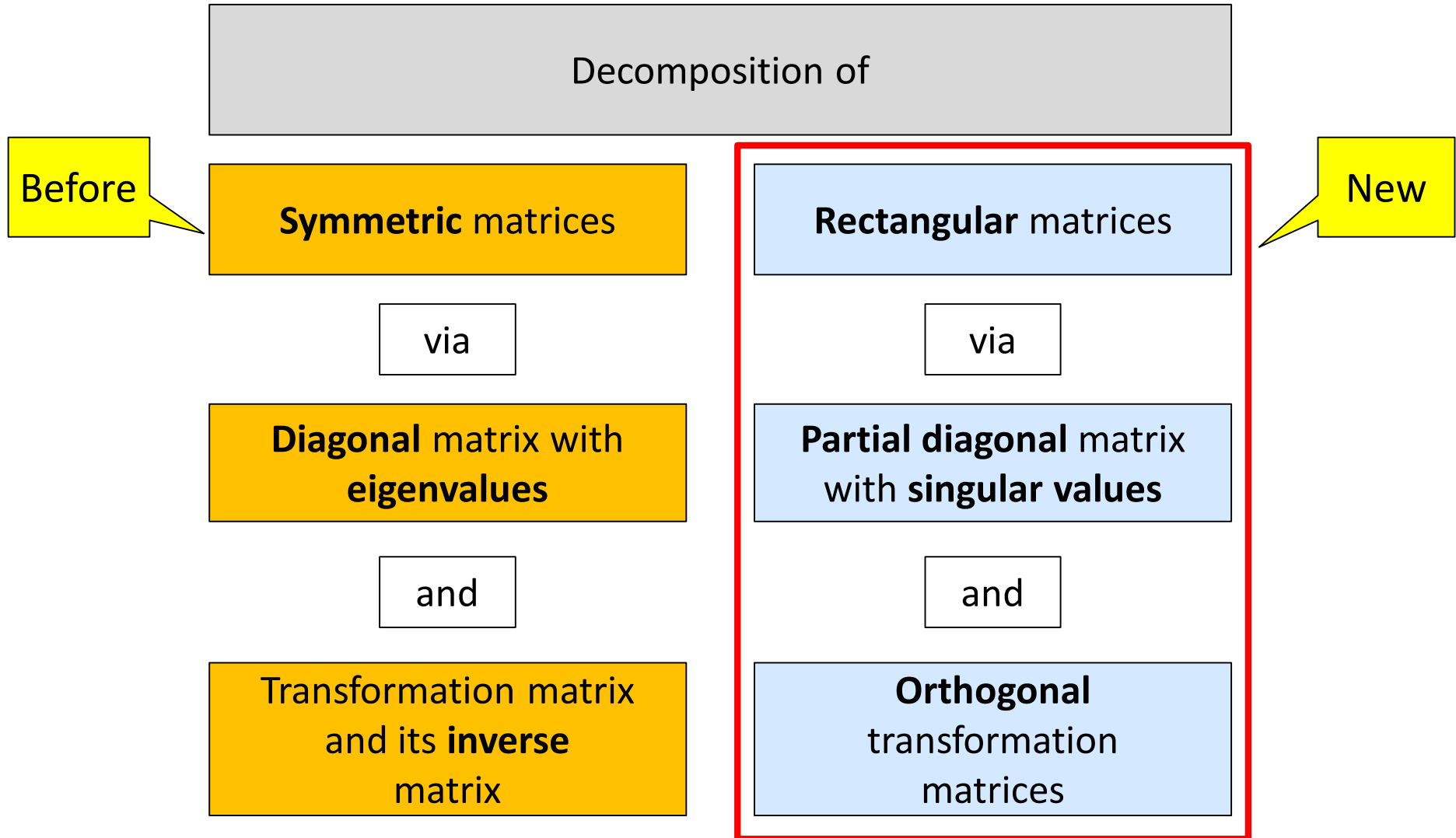
General Idea

A **decomposition via a diagonal matrix** is extremely helpful as it simplifies a lot of computations.

Hence one might ask, if this concept which applies to quadratic matrices can **somehow also be used for rectangular matrices**.

This is possible, involves however a bit **more details**.
One needs the concepts of **rank** and **orthogonal matrices**.

Introduction



Preparation - Rank

The **rank of a matrix** is the number of **columns that are not a linear combination of each other** (= number of „linearly independent“ column vectors).

Examples:

The matrix A has rank two as there are only two linearly independent column vectors.

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 4 \\ 0 & 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \neq x * \begin{pmatrix} 0 \\ 4 \\ 0 \end{pmatrix}$$

The matrix B has rank one as the column vectors are a multiple of each other.

$$B = \begin{pmatrix} 4 & 0 & 2 \\ 2 & 0 & 1 \end{pmatrix}, 0,5 * \begin{pmatrix} 4 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

However, the matrix C has also rank two, as one can get the third column by linear combination.

$$C = \begin{pmatrix} 1 & 0 & 3 \\ 0 & 4 & 16 \\ 0 & 0 & 0 \end{pmatrix}, 3 * \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} + 4 \begin{pmatrix} 0 \\ 4 \\ 0 \end{pmatrix} = \begin{pmatrix} 3 \\ 16 \\ 0 \end{pmatrix}$$

The first two columns are however not a combination of each other.

Exercise

What is the rank of

1) $\begin{pmatrix} 1 & 3 & 3 \\ 0 & 11 & 11 \\ 0 & 0 & 1 \end{pmatrix}$

2) $\begin{pmatrix} 4 & 0 & 2 \\ 0 & 0 & 1 \end{pmatrix}$

3) $\begin{pmatrix} 11 & -6 \\ 18 & -10 \end{pmatrix}$

4) $\begin{pmatrix} 4 & 3 & 2 \\ 0 & 0 & 0 \end{pmatrix}$

Preparation – Normalized vectors

Vectors which have a norm of **norm of 1** are **normalized**.

And each vector can be normalized by multiplying it with an appropriate factor.

Example:

$x = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ has a norm of $\|x\| = \sqrt{2^2 + 1^2} = \sqrt{5}$, so it is not normalized.

But $\frac{1}{\sqrt{5}}x = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ is normalized as $\left\| \frac{1}{\sqrt{5}}x \right\| = \left| \frac{1}{\sqrt{5}} \right| \|x\| = \frac{1}{\sqrt{5}} \sqrt{5} = 1$

Exercise

Are the following vectors normalized? If not, normalize them.

a) $x = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$

b) $y = \frac{1}{\sqrt{50}} \begin{pmatrix} 7 \\ 1 \end{pmatrix}$

c) $z = \frac{1}{\sqrt{35}} \begin{pmatrix} 5 \\ 3 \end{pmatrix}$

Preparation – Orthogonal vectors

Two **vectors** are called **orthogonal** to each other if their **scalar product equals zero**.

Scalar product is a **special kind** of matrix product:

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix}^T \begin{pmatrix} 3 \\ 4 \end{pmatrix} = 1 * 3 + 2 * 4$$

Vectors are called „**orthonormal**“, if they are orthogonal **and** normalized.

Example:

The vectors $\begin{pmatrix} 8 \\ -2 \end{pmatrix}$ and $\begin{pmatrix} 1 \\ 4 \end{pmatrix}$ are orthogonal since

$$\begin{pmatrix} 8 \\ -2 \end{pmatrix}^T \begin{pmatrix} 1 \\ 4 \end{pmatrix} = 8 * 1 - 2 * 4 = 0$$

Exercise

Are the following vectors orthogonal?

a) $\begin{pmatrix} 7 \\ -3 \end{pmatrix}$ and $\begin{pmatrix} -1 \\ 5 \end{pmatrix}$

b) $\begin{pmatrix} -5 \\ -2 \end{pmatrix}$ and $\begin{pmatrix} -4 \\ 10 \end{pmatrix}$

Preparation – Orthogonal matrices

In order to handle the following decompositions of a matrix, we introduce a new type of matrix namely an **orthogonal** matrix O which has the defining property

$$O^T O = O O^T = I$$

meaning that the matrix multiplied with its transpose (or vice versa) yields the identity matrix. In the following it will play the role of an inverse matrix.

Example:

$$O = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

is an orthogonal matrix since

$$O^T O = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} * \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$O O^T = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} * \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

How to create a symmetric matrix from an arbitrary matrix?

Given an arbitrary matrix A , try $A^T A$.

Why?

Check $(A^T A)^T$.

According to the rules before:

$$(A^T A)^T = A^T A^{TT} = A^T A$$

Singular value decomposition

The already mentioned eigenvalue decomposition applies (at least) to symmetric matrices. As this decomposition exhibits many useful properties, one typically would like to **extend** such a **decomposition** to non-symmetric / **rectangular matrices**. In fact, such a decomposition can be recovered for rectangular $m \times n$ matrices. We use matrix $\delta_{\mu\sigma}$ where diagonal entries are 1, otherwise 0.

Given a $m \times n$ matrix A , a decomposition

$$A = USV^T$$

with U being a $m \times m$ orthogonal matrix, V being a $n \times n$ orthogonal matrix and S being a diagonal $m \times n$ matrix $S = (\sigma_\mu \delta_{\mu\sigma})$, for example

$$\begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{pmatrix},$$

is called a **singular value decomposition**. The σ_μ are the **singular values**.

Singular value decomposition

More precisely:

Given a $m \times n$ matrix A with rank r . Let $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \dots \geq \lambda_r > 0 = \lambda_{r+1} = \dots = \lambda_n$ be the eigenvalues of $A^T A$ with a corresponding system of orthonormal eigenvectors $v^1, v^2, \dots, v^n$.

Then

$$u^v := \frac{1}{\sigma_v} A v^v \quad \text{with} \quad \sigma_v := +\sqrt{\lambda_v}, 1 \leq v \leq r$$

constitutes an orthonormal system of eigenvectors for AA^T for the eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \dots \geq \lambda_r$ which can be complemented to an orthonormal system $u^1, u^2, \dots, u^m$ of eigenvectors for AA^T .

Singular value decomposition

More precisely:

By setting $V = (v^1, v^2, \dots, v^n)$, $U = (u^1, u^2, \dots, u^m)$ and $S = (\sigma_\mu \delta_{\mu\nu})$ with $\sigma_\mu := +\sqrt{\lambda_\mu}$ for $1 \leq \mu \leq r$ and $\sigma_{r+1} = \sigma_{r+2} = \dots = \sigma_{\min(m,n)} = 0$, then A possesses the singular value decomposition

$$A = USV^T$$

with singular values $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \dots \geq \sigma_r > 0$.

Singular value decomposition: Example (1/3)

Consider $A = \begin{pmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{pmatrix}$.

First we form $A^T A$ which is $\begin{pmatrix} 17 & 8 \\ 8 & 17 \end{pmatrix}$. Getting the respective eigenvalues we consider (as before)

$$\begin{vmatrix} 17 - \gamma & 8 \\ 8 & 17 - \gamma \end{vmatrix} = (17 - \gamma) * (17 - \gamma) - 8 * 8 =$$
$$= \gamma^2 - 34\gamma + 225 = (\gamma - 25)(\gamma - 9) = 0$$

Hence the singular values are $\sigma_1 = +\sqrt{25} = 5$ and $\sigma_2 = +\sqrt{9} = 3$.

Singular value decomposition: Example (2/3)

We still need the orthonormal eigenvalues for forming $V = (v^1, v^2, \dots, v^n)$ and $U = (u^1, u^2, \dots, u^m)$,
Some computations (finding eigenvectors as before and normalizing) then yield

$$U = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{18}} & -\frac{1}{\sqrt{18}} & \frac{4}{\sqrt{18}} \\ \frac{2}{3} & -\frac{2}{3} & -\frac{1}{3} \end{pmatrix}$$

and

$$V^T = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}$$

Singular value decomposition: Example (3/3)

Hence the full singular decompositions reads

$$A = USV^T$$

$$\begin{pmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{18}} & \frac{2}{3} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{18}} & -\frac{2}{3} \\ 0 & \frac{4}{\sqrt{18}} & -\frac{1}{3} \end{pmatrix} \begin{pmatrix} 5 & 0 \\ 0 & 3 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}$$

Interesting question:

About the SVD (singular value decomposition):

Why do we have a **different number** of u 's and v 's?

As claimed, we start with the v 's (here only **two**!) from which we build the u 's (as defined).

But why do we then finally have **three** u 's?

The u 's constitute an orthonormal system of eigenvectors for $A A^T$.

The point is: for any starting two vectors we can exactly determine a third one such that it constitutes an orthonormal system of eigenvectors (Algorithm: **Gram-Schmidt-orthogonalization**).

Singular value decomposition

One might immediately ask how the singular values of a matrix are related to the eigenvalues of a matrix, since the singular values can somehow be thought of as a „replacement“ for them.

In fact, the **squares** of the **singular values** of $\mathbf{A}$ are the **eigenvalues** of $\mathbf{A}^T \mathbf{A}$.

Or in other words, the singular values of $\mathbf{A}$ are the roots of the eigenvalues of $\mathbf{A}^T \mathbf{A}$.

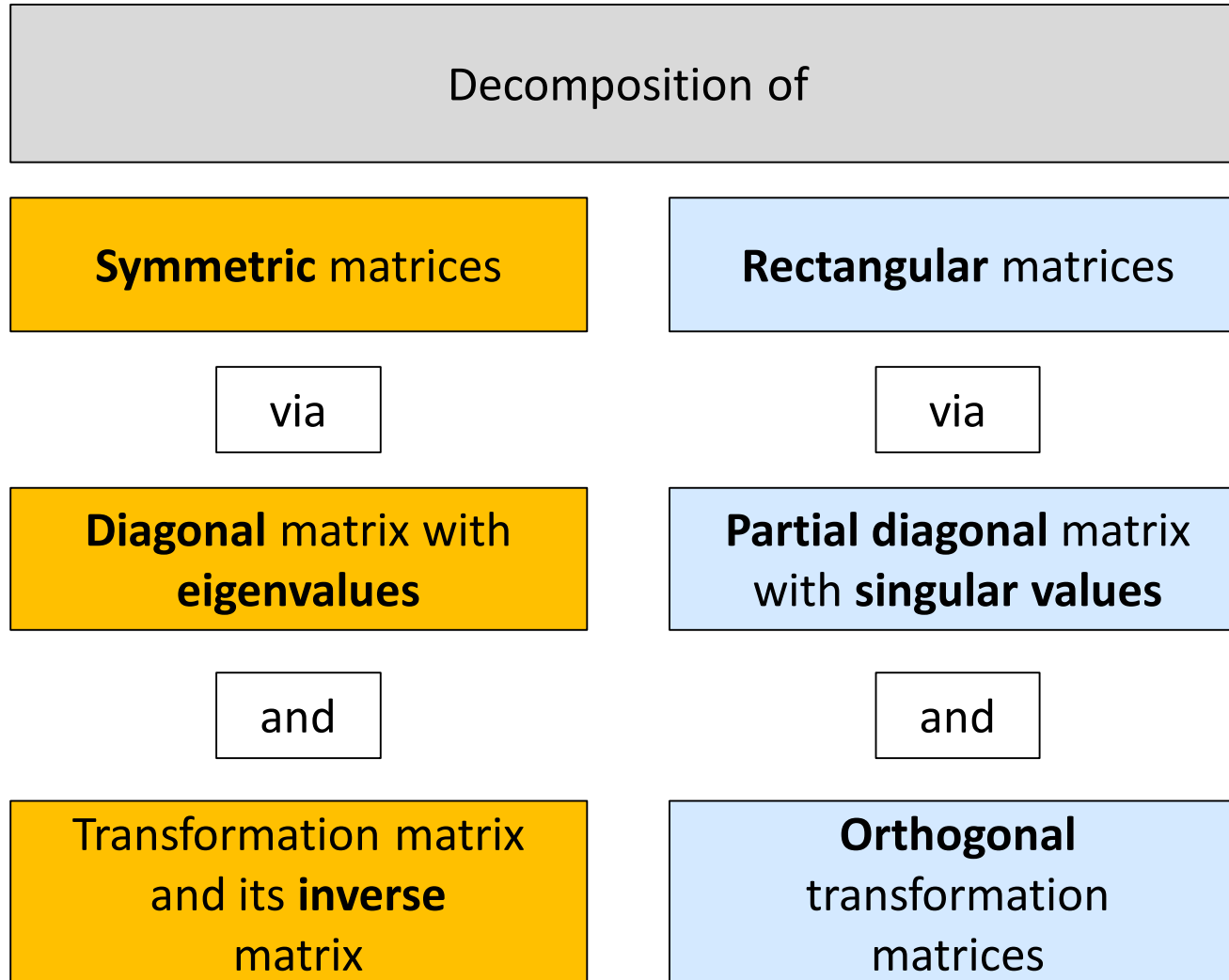
In the following we will apply the singular value decomposition to ill-posed systems of linear equations.

Exercise

a) Find the singular value(s) of $A = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}$.

b) Find the singular value(s) of $B = \begin{pmatrix} 2 & 0 \\ 0 & 3 \\ 0 & 0 \end{pmatrix}$.

Summary



01

Numerical Mathematics

2.1 Computing

2.2 Eigenvalues and Eigenvectors II

2.3 Systems of Linear Equations II

2.4 Approximation

2.5 Iterative Approaches

Error estimation

In practical applications which lead to a system of linear equations ($Ax=b$) the matrix A or the vector b may **not exactly** be **given** due to imprecisions.

Furthermore, the **algorithms** for solution **contribute further to** the already existing **imprecision** (especially when the system of linear equations is huge).

Therefore, it is crucial that the **resulting error** can be **estimated correctly**.

Condition of a matrix

In estimating the above mentioned error, the concept of **condition** of a matrix will be extremely helpful. With this we can systematically assess the implications of noise terms in A and b .

Condition of a matrix

Let an invertible matrix A be given. Instead of considering $Ax=b$, we consider the diluted equation

$$A(x + \Delta x) = b + \Delta b$$

with Δb being the original noise. We can compute the corresponding noise in the solution vector x by

$$\Delta x = A^{-1}\Delta b$$

Using this relation in an (arbitrary) vector norm (with induced Matrix norm), we get

$$\|\Delta x\| \leq \|A^{-1}\| * \|\Delta b\|$$

By using $\|b\| \leq \|A\| * \|x\|$ in the equation above, we finally get for $b \neq 0$

$$\frac{\|\Delta x\|}{\|x\|} \leq \|A^{-1}\| * \|A\| * \frac{\|\Delta b\|}{\|b\|}$$

So $\|A^{-1}\| * \|A\|$ somehow measures the sensitivity of the relative error of x („error amplification“).

Condition of a matrix

Hence, we make the following definition: For an invertible matrix A its **condition**, $\text{cond}(A)$, is defined as

$$\text{cond}(A) := \|A^{-1}\| * \|A\|$$

The condition of a matrix is always at least 1 (just try $1 = \|I\| = \|A^{-1} * A\| \leq \|A^{-1}\| * \|A\|$).

With this „*measurement of error amplification*“ we can **precisely estimate the resulting error** from initial noise (some computations):

Given a matrix A with noise ΔA and the solution x with its noise Δx , satisfying $Ax=b$ and $(A + \Delta A)(x + \Delta x)=b$, $b \neq 0$. Assume further that $\|\Delta A\| * \|A^{-1}\| < 1$. Then

$$\frac{\|\Delta x\|}{\|x\|} \leq \frac{\text{cond}(A)}{1 - \text{cond}(A) \frac{\|\Delta A\|}{\|A\|}} * \frac{\|\Delta A\|}{\|A\|}$$

Pseudonormal solutions

Up to now we have considered systems of linear equations

$$Ax = b$$

where a $n \times n$ matrix A maps a n vector x to a n vector b .

However, one can consider the problem in a **more general context**, with a $m \times n$ matrix A maps a m vector x to a n vector b . The case of

- $m > n$ is referred to as an **overdetermined** system of equations,
 - whereas the case $m < n$ is referred to as an **underdetermined** system of equations
- (meaning there there are either too much or too few defining equations to reach an unique solution).

Regardless of the case at hand, one defines a corresponding **minimization problem** as

$$\|A\tilde{x} - b\| = \inf_x \|Ax - b\|$$

Pseudonormal solutions

The already introduced **singular value decomposition** yields an opportunity to **directly give** the **solutions** $\tilde{x}$ of the minimization problem.

Assume a singular value decomposition of A as before: $A = USV^T$. Inserting this and using that orthogonal matrices do not change the norm yields

$$\|Ax - b\|^2 = \|U^T(Ax - b)\|^2 = \|SV^Tx - U^Tb\|^2 = \|Sz - d\|^2$$

using $z := V^Tx$ and $d := U^Tb$. Now we can **explicitly give** the **solution** of the minimization problem (recall that S has a diagonal part in $1, 2, \dots, r$ and a lot of zeros for $r + 1, \dots, n$) via

- $z_\mu = \frac{1}{\sigma_\mu} d_\mu$ for $\mu = 1, 2, \dots, r$
- z_μ arbitrary for $\mu = r + 1, \dots, n$

Pseudonormal solutions

By the derivation above, every **solution** $\tilde{x}$ of the **minimization problem** can be written as

$$\tilde{x} = \sum_{\mu=1}^r \frac{1}{\sigma_{\mu}} d_{\mu} v^{\mu} + \sum_{\mu=r+1}^n z_{\mu} v^{\mu}$$

The last $n-r$ columns of the matrix V are mapped to zero by $A^T A$, meaning that their contributions to the solution are not crucial. Hence **only the first sum is essential** for solving the minimization problem which leads to the following definition/conclusion:

$$x^{+} = \sum_{\mu=1}^r \frac{1}{\sigma_{\mu}} d_{\mu} v^{\mu}$$

is the **pseudonormal solution** of the minimization problem $\|A\tilde{x} - b\| = \inf_x \|Ax - b\|$. Its **vector norm** $\|\tilde{x}\|$ is the **smallest** of all possible solutions of the minimization problem (all other solutions have an additional non-essential part which only „inflates“ the norm).

Pseudonormal solutions - Example

We try to solve

$$\begin{pmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{pmatrix} * \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \\ 1 \end{pmatrix}$$

This is not possible! (overdetermined system of linear equations)

Applying the approach from before we find the pseudonormal solution $\begin{pmatrix} 1,22 \\ 0,78 \end{pmatrix}$.

Inserting it yields $\begin{pmatrix} 5,2 \\ 4,8 \\ 0,9 \end{pmatrix}$ which is closest in the norm to $\begin{pmatrix} 5 \\ 5 \\ 1 \end{pmatrix}$ of all “non-solutions”.

Excercise

What is the computation for the pseudonormal solution?

Solution

$$\mathbf{x}^+ = \sum_{\mu=1}^r \frac{1}{\sigma_{\mu}} d_{\mu} v^{\mu}$$

So using the SVD from before, we obtain

$$\mathbf{x}^+ = \frac{1}{\sigma_1} d_1 v^1 + \frac{1}{\sigma_2} d_2 v^2$$

Remember that $d = U^T b$. Hence

$$\mathbf{x}^+ = \frac{1}{5} \left(\frac{5}{\sqrt{2}} + \frac{5}{\sqrt{2}} \right) * \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 1 \\ \frac{1}{\sqrt{2}} \end{pmatrix} + \frac{1}{3} \left(\frac{5}{\sqrt{18}} - \frac{5}{\sqrt{18}} + \frac{4}{\sqrt{18}} \right) * \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix} = \begin{pmatrix} 1,22 \\ 0,78 \end{pmatrix}$$

A more general/explicit expression for the condition of a matrix

Once again we try to solve a system of linear equations $Ax = b$. With help of the singular value decomposition and pseudonormal solutions one can derive (with some computations) the relation

$$\text{cond}(A) = \frac{\sigma_1}{\sigma_r}$$

Hence, we can not only describe the condition of a quadratic matrix more precisely, but even do so for arbitrary rectangular matrices A with a given singular value decomposition.

Summarizing:

Given a rectangular matrix A with singular value decomposition $A = USV^T$, then the condition of the matrix can be computed as the quotient of the biggest and smallest singular value:

$$\text{cond}(A) = \frac{\sigma_1}{\sigma_r}$$

Regularization of a system of linear equations

Now what did we get from introducing the singular value decomposition of a matrix?

By the insights that we have gained, we can **improve** the **condition** of a **matrix** which then leads to a **computationally more stable solution** of the system of linear equations.

We have noticed that the singular values play a crucial role in the computation of the condition. Therefore, one might get the idea that an „**adjustment**“ of these **singular values** would be a helpful way for proceeding.

We will now consider how this „adjustment“ is made.

Regularization of a system of linear equations

We have noticed that the condition of a matrix is directly determined by its singular values via

$$\text{cond}(A) = \frac{\sigma_1}{\sigma_r}$$

Therefore one could try to improve this condition by the following **adjustment** of the singular values (from $A = USV^T$).

One adjusts S by omitting singular values which are smaller than a given threshold τ (because then division by higher singular values yield a smaller condition):

$$S_{\tau}^{+} := (\eta_{\mu} \delta_{\mu\nu}), \eta_{\mu} := \begin{cases} \sigma_{\mu}^{-1} & \text{if } \sigma_{\mu} \geq \tau \\ 0 & \text{else} \end{cases}$$

Then one puts all parts together as $A_{\tau}^{+} = US_{\tau}^{+}V^T$.

Regularization of a system of linear equations

Now using $A_{\tau}^{+} = U S_{\tau}^{+} V^T$ yields a problem which is **conditioned much better**.

Due to the approximation (A vs. A_{τ}^{+}) there is of course a little error,
but it is **small compared** to bigger errors that arise out of the bad condition of the initial problem.

Exercise

Assume that you have a diluted system of linear equations

$$A(x + \Delta x) = b + \Delta b$$

The biggest and smallest singular value of A are 10 and 2.

Give an upper bound for the relative error of x ($\frac{\|\Delta x\|}{\|x\|}$) if the relative error of b ($\frac{\|\Delta b\|}{\|b\|}$) is 4%.

01

Numerical Mathematics

- 2.1 Computing
- 2.2 Eigenvalues and Eigenvectors II
- 2.3 Systems of Linear Equations II
- 2.4 Approximation**
- 2.5 Iterative Approaches

Uses of Approximations

For many machine learning applications like reinforcement learning one needs suitable approximations of functions (**representing signals**).



For these function approximations one uses advanced mathematical methods like

- **Polynomials**
- **Fourier Bases**
- **Coarse Coding**

for which we now provide more details.

Approximation Method - Polynomials

The **most simple method** of approximation consists in using **polynomials** $p(x)$.

Examples for polynomials dependent on one variable x

(with different order, w_i representing the weights that are to be found by MLprocedure):

- Linear $p(x) = w_1x + w_0$
- Quadratic $p(x) = w_2x^2 + w_1x + w_0$
- Cubic $p(x) = w_3x^3 + w_2x^2 + w_1x + w_0$
- ...

Dependent on the function that shall be approximated one chooses the appropriate order of polynomials.

Approximation Method - Polynomials

Assume for example that a reinforcement learning problem has two numerical dimensions (which refer to example to speed and altitude).

Naively, one could try to represent the state of the agent by a two-dimensional vector (x, y) .

However, this only represents the separated dimensions and **interaction effects** between the two dimensions are **not apparent**. Therefore, one might come up with a longer vector $(1, x, y, xy)$ where we already have an **interaction term** xy (for example speed increase because of a nosedive).

If **more complex interactions** not to be taken into account, one might even choose a longer vector like $(1, x, y, xy, x^2, y^2, x^2y, xy^2, x^2y^2)$.

Now the weights w_i will be found by the machine learning procedure.

Approximation Method - Polynomials

Now why could we try to use polynomials for approximation?

It is actually known (big theorem) that any continuous function (in a closed interval) can be approximated „arbitrarily precise“ by polynomials. That is why one can legitimately try to use easy polynomials to approximate any given continuous signal.

Sketch

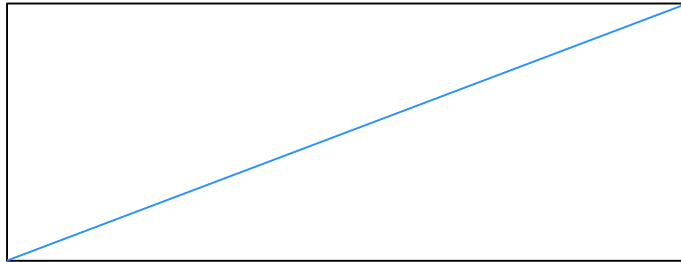
Of course, the higher the order of the polynomial, the better the approximation, but then a lot more weights w_i need to be found.

Approximation Method - Polynomials

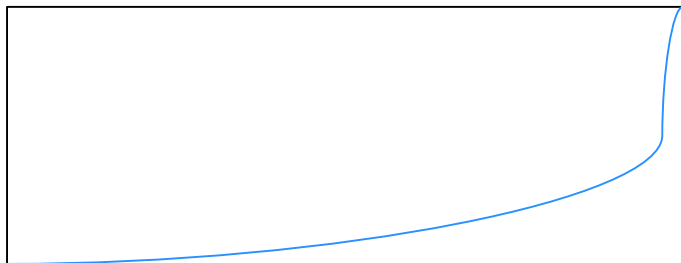
Exercise

By which polynomial order would you try to approximate the following signals:

a)



b)



Approximation Method – Fourier Bases

The theorem that guarantees the feasibility of polynomial approximation makes a special assertions for **periodic functions**.

It states that these periodic functions can be approximated „arbitrarily precise“ by trigonometric polynomials. Trigonometric polynomials are polynomials where the variable x is a trigonometric function (sine or cosine).

Examples (similar to the ones before):

- Linear $p(x) = w_1 \sin(x) + w_0$
- Quadratic $p(x) = w_2 \sin(x)^2 + w_1 \cos(x) + w_0$
- Cubic $p(x) = w_3 \cos(x)^3 + w_2 \sin(x)^2 + w_1 \cos(x) + w_0$
- ...
- Or even mixed: $p(x) = w_2 \sin(x) \cos(x) + w_1 \cos(x) + w_0$
- ...

Approximation Method – Fourier Bases

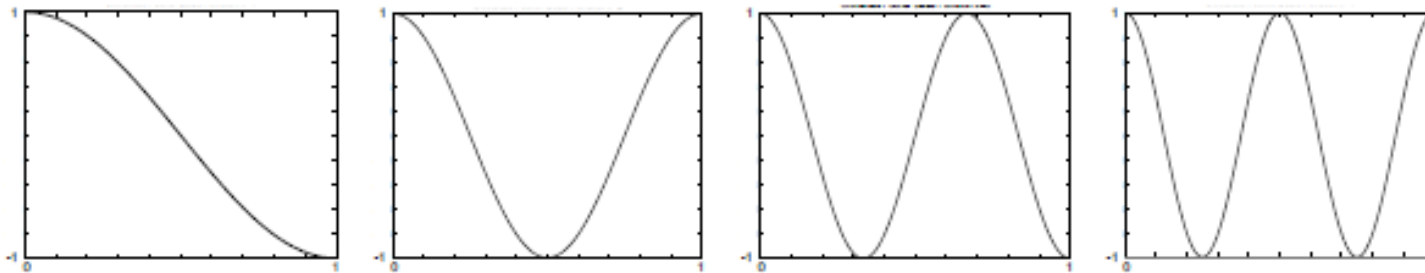
How does this help us exactly?

By cleverly composing trigonometric polynomials, one can set the **frequency of fluctuations**:

For example by $2\sin(x)\cos(x) = \sin(2x)$ which means that the frequency of fluctuations is doubled.

Generally one can, by such a clever combination, produce any frequency of fluctuations:

$\sin(nx)$ or $\cos(nx)$



The overlay of such functions can then, as stated, produce any periodic function.

Approximation Method – Fourier Bases

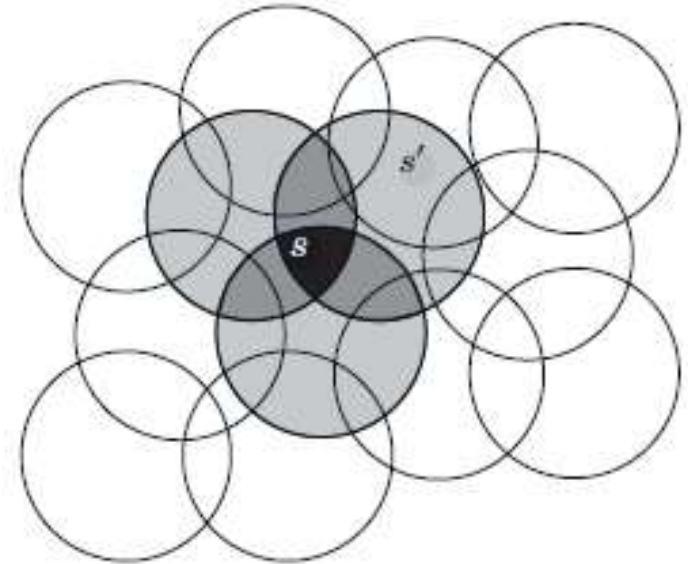
Again as before, to approximate a given complex function one might need a lot of different powers of sines and cosines with respective weight. This increases of course the computational effort.

So one has to decide between **better approximation** and **higher computational effort vs. suboptimal approximation** and **lower computational effort**.

Approximation – Coarse Coding

In the case of a two-dimensional space, the idea is to cover the space with circles.

If two states are in the **same circle**, they **share** a certain **feature**. Of course two states can lie in more than one encompassing circle, meaning that they share more than one feature.



In the picture on the right, the states s and s' just lie in one common circle, meaning that they share only one feature. Hence, an approximation of s by s' would be rather weak.

Approximation – Coarse Coding

Exercise

Coarse Coding works by putting states that share a feature in the same circle.

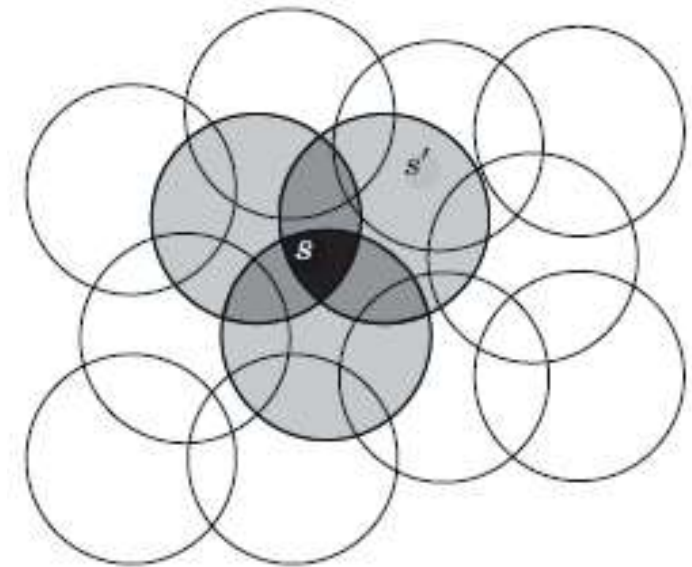
- When doing such an approximation, what could be **major influencing factors**?
- Which of those should have a **larger influence**?

Approximation – Coarse Coding

For machine learning applications each circle corresponds to a weight w_i (recall that finding the best weights is the standard objective of each machine learning procedure).

If the **circles** are **large**, one gets **broad approximations** (or generalizations), whereas if the **circles** are **small** one gets **narrow approximations**.

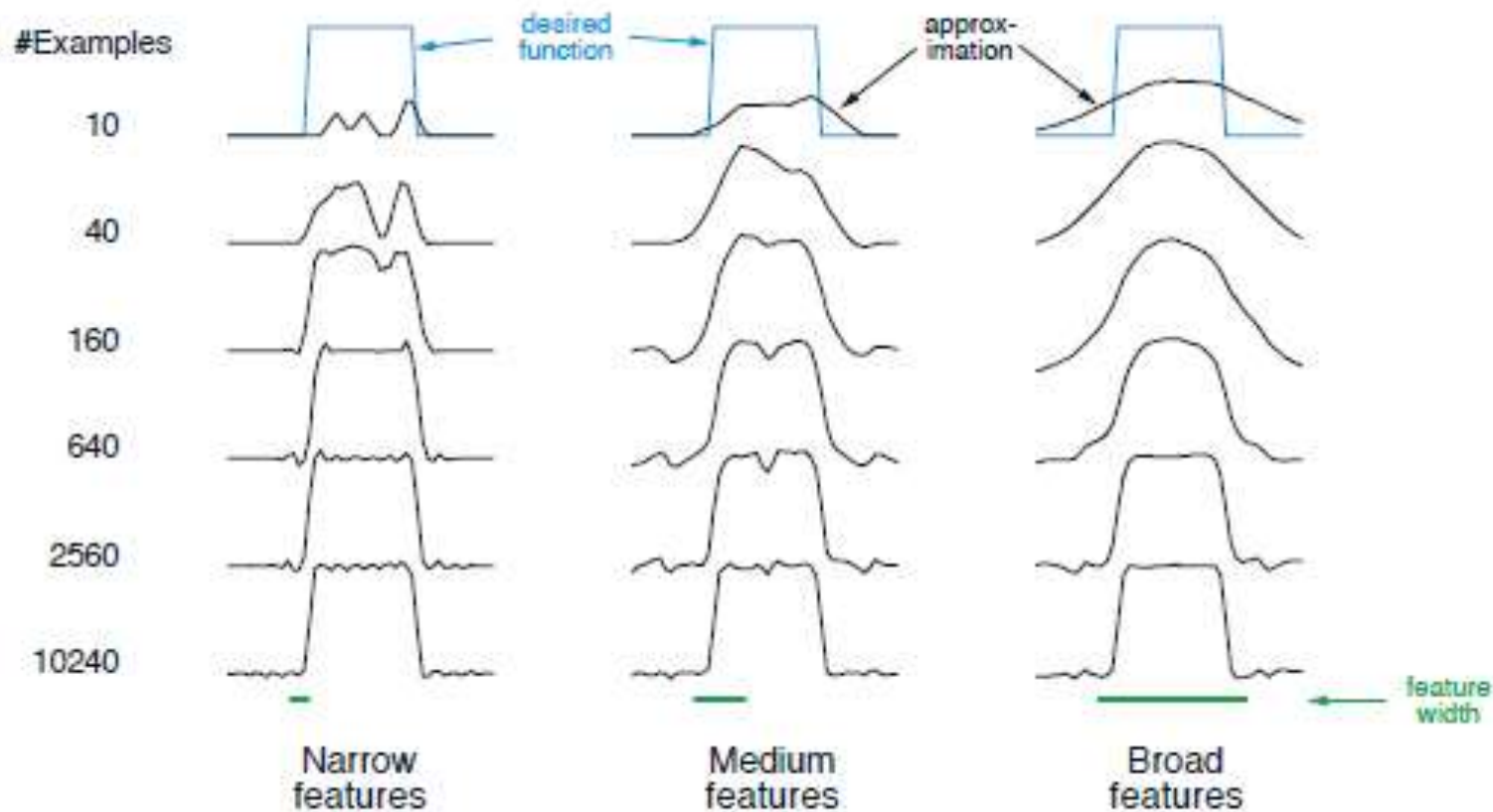
This is one influencing factor, but of course **also** the **number of circles plays an important role**.



2.4 Approximation

Approximation – Coarse Coding

However it turns out, that the **number** of circles is much **more important than** the **size** of the circles:



01

Numerical Mathematics

- 2.1 Computing
- 2.2 Eigenvalues and Eigenvectors II
- 2.3 Systems of Linear Equations II
- 2.4 Approximation
- 2.5 Iterative Approaches**

General Idea

Often, one tries to find the correct input for a function f (f can be any sales, profit or NPV function) for getting a target value y : **Which input** value x **yields** the **target output** $y = f(x)$?

However, it might be that finding the exact solution is very complex or practically not feasible (in terms of computational effort).

That is why **iterative approaches** are used which approximate the correct solution by „small steps“.

Even if they do not reach the exact solution, it might be that the iterative solution is already **good enough**. Therefore, it is helpful to derive certain **bounds for the remaining imprecision**.

In the following, iterative approaches and their error bounds will be explained.

Rephrasing of the problem

For reasons that will become apparent in the next slides, we do not explicitly consider the function f , but the **modified function**

$$\mathbf{F} := f - y + \mathbf{id}$$

with $\mathbf{id}$ being the identity function.

Why? Since then the original problem

is equivalent to

$$y = f(x)$$

$$\mathbf{F}(\mathbf{x}) = \mathbf{x}$$

(Just check by plugging in: $\mathbf{F}(\mathbf{x}) = f(x) - y + x = 0 + x = x$)

Analyzing the rephrased problem

So we have replaced the original problem with a **fixed point equation** since the **input x gives the same output**:

$$F(x) = x$$

Now the iterative procedure consists in **successively applying** the newly formed function **F**, in the sense that **each application of it provides an iteration**:

$$x^{k+1} = F(x^k)$$

The starting point will be denoted as x^0 .

Nice idea, but how can this help us exactly?

Analyzing the rephrased problem

How can this help us exactly?

If the function F satisfies a **certain condition**, then the iteration $x^{k+1} = F(x^k)$ **automatically approaches the right solution**. It might not reach it exactly, but getting there with high precision.

The required condition is that the function F is a **contraction** meaning that for any a and c

$$\|F(a) - F(c)\| \leq \alpha \|a - c\|$$

for some $\alpha < 1$ (in a given norm $\| \cdot \|$). This means that each application of F **reduces the original distance** (sketch!).

Analyzing the rephrased problem

Example:

$$F(x) := \frac{1}{2}x + \frac{1}{x}$$

Notice that the equation

$$F(x) = x$$

reads

$$\frac{1}{2}x + \frac{1}{x} = x$$

which is equivalent to

$$x^2 + 2 = 2x^2 \quad \text{or} \quad x^2 = 2$$

This means we want to get the root of 2!

Analyzing the rephrased problem

Example:

$$F(x) := \frac{1}{2}x + \frac{1}{x}$$

Then we first check whether this function is contracting ($a, c \in [1,2]$):

$$\begin{aligned}\|F(a) - F(c)\| &= \left\| \frac{1}{2}a + \frac{1}{a} - \frac{1}{2}c - \frac{1}{c} \right\| = \left\| \frac{1}{2}(a - c) - \frac{1}{ac}(a - c) \right\| = \\ &= \left\| \frac{1}{2} - \frac{1}{ac} \right\| * \|a - c\| \leq \frac{1}{2} * \|a - c\|\end{aligned}$$

Analyzing the rephrased problem

Example:

Since we have found an $\alpha = \frac{1}{2} \leq 1$, the function is contracting and we can find a solution to the equation $\frac{1}{2}x + \frac{1}{x} = x$ (or equivalently $x^2 = 2$) iteratively.

In consequence, one can **compute $\sqrt{2}$ by pure iteration** with an arbitrary starting point.

	F(previous value)	F(previous value)	F(previous value)	F(previous value)
Start				
0,5	2,25	1,56944444	1,42189036	1,41423429
7	3,64285714	2,09593838	1,52508245	1,41824348

Exercise

Approximate $\sqrt{1,25}$ by pure iteration (starting point 5, four iterations).

Application to systems of linear equations

Can we apply the **same method** to **systems of linear equations**?

As usual, we want to solve

$$Ax = b$$

for A a quadratic matrix.

Now we apply our learnings from before. The first step will be the rephrasing of the equation to

$$x = x + 0 = x - Ax + b = (I - A)x + b = x \quad \mathbf{F(x) = x}$$

with I being the identity matrix.

Now we set

$$C := I - A$$

hence

$$F(x) := Cx + b$$



Application to systems of linear equations

Again, we can apply

$$x^{k+1} = F(x^k)$$

after we have checked that the function F is a contraction. But how do we check it for matrices?

Checking that the function F is a contraction:

$$\begin{aligned}\|F(a) - F(c)\| &= \|Ca + b - (Cc + b)\| = \\ &= \|Ca - Cc\| = \|C(a - c)\| \leq \|C\| * \|a - c\|\end{aligned}$$

Here it would be sufficient that $\|C\| < 1$.

One can check it if the the **largest absolute eigenvalue** (the „spectral radius“ $\rho(C)$) is ≤ 1 .

For **special C matrices**, we have **even easier conditions** (next slides)!

Application to systems of linear equations – Special Case

Let A be an invertible matrix $n \times n$, then decompose A via

$$A = -L + D - R$$

with the following $n \times n$ matrices

$$L := - \begin{pmatrix} 0 & & 0 \\ a_{21} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ a_{n1} & a_{nn-1} & 0 \end{pmatrix}$$

$$R := - \begin{pmatrix} 0 & a_{12} & a_{1n} \\ \vdots & \ddots & \vdots \\ 0 & \cdots & a_{n-1n} \\ 0 & \cdots & 0 \end{pmatrix}$$

$$D := \begin{pmatrix} a_{11} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & & 0 \\ 0 & \cdots & a_{nn} \end{pmatrix}$$

Application to systems of linear equations – Special Case

The idea is to **prepare** the **iteration matrix** C via a **division** by the **diagonal elements**.
Starting with the decomposition from before:

$$Ax = (-L + D - R)x = b$$

Now applying D^{-1} :

$$D^{-1}Ax = D^{-1}(-L + D - R)x = (-D^{-1}L + I - D^{-1}R)x = D^{-1}b$$

Summarized ($A' := D^{-1}A$):

$$A'x = D^{-1}b$$

Then as before:

$$F(x) := (I - A')x + D^{-1}b = \boxed{D^{-1}(L + R)}x + D^{-1}b$$

Summarized ($b' := D^{-1}b$):

$$F(x) := \boxed{C}x + b'$$

For this new C matrix the contraction condition can be checked more easily! (some computations)

Application to systems of linear equations – Special Case

Now for this type of decomposition one can formulate a very easy check that the iterations

$$F(x) := Cx + b'$$

approach the right solution.

Criterion

Let a quadratic matrix A be given and further suppose that

$$\sum_{\substack{v=1 \\ v \neq \mu}}^n |a_{\mu v}| < |a_{\mu \mu}|$$

for $1 \leq \mu \leq n$ (meaning that the diagonal entries dominate).

Then the iteration approaches automatically the correct solution.

Example

You are trying to solve a system of linear equations $Ax = b$ with invertible matrix A .

Assume that A is decomposed via $A = -L + D - R$ and that we apply the preparation steps from before (preparing the iteration matrix via a division by the diagonal elements). Accordingly we use the respective iterations

$$F(x) := Cx + b'$$

For

$$A = \begin{pmatrix} 7 & -5 \\ 9 & -10 \end{pmatrix}$$

we can directly infer that the iterative procedure automatically approaches the direct solution.

Counter-Example

This also functions for our initial hot tub example:

$$\begin{pmatrix} 1 & 2 \\ 2 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 8 \\ 18 \end{pmatrix}$$

Of course $|a_{12}| = 2 < 1 = |a_{11}|$, is not fulfilled, so the necessary criterion for our previous convergence theorem is not satisfied.

However, after some iterations (approx. 30) with start $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ yield $\begin{pmatrix} 4,05 \\ 2,03 \end{pmatrix}$ which is very close to the precise solution $\begin{pmatrix} 4 \\ 2 \end{pmatrix}$.

Exercise

What are C and b' ?

Exercise

You are trying to solve a system of linear equations $Ax = b$ with invertible matrix A .

Assume that A is decomposed via $A = -L + D - R$ and that we apply the preparation steps from before (preparing the iteration matrix via a division by the diagonal elements). Accordingly we use the respective iterations

$$F(x) := Cx + b'$$

Can we directly infer that the iterative procedure automatically approaches the direct solution for the following matrices?

$$1) \begin{pmatrix} 5 & 1 & 0 \\ 2 & 7 & 3 \\ 1 & 0 & 2 \end{pmatrix}$$

$$2) \begin{pmatrix} 5 & 2 & 2 \\ 1 & 4 & 3 \\ 1 & 3 & 8 \end{pmatrix}$$



Thank you for attending this course.